

KSPMI: A Knowledge-based System for Predictive Maintenance in Industry 4.0

Qiushi Cao^{a,*}, Cecilia Zanni-Merk^b, Ahmed Samet^c, Christoph Reich^d, François de Bertrand de Beuvron^c, Arnold Beckmann^a and Cinzia Giannetti^e

^aDepartment of Computer Science, Swansea University, SA1 8EN Swansea, United Kingdom

^bLITIS, Normandie Université/INSA Rouen, 76000 Saint-Étienne-du-Rouvray, France

^cICUBE/CSIP, INSA de Strasbourg, 24 Boulevard de la Victoire, 67084 Strasbourg, France

^dIDACUS Institute of Data Science, Cloud Computing and IT Security, Hochschule Furtwangen University, 78120 Furtwangen, Germany

^eThe Future Manufacturing Research Institute (FMRI), Faculty of Science and Engineering, Swansea University, SA1 8EN Swansea, United Kingdom

ARTICLE INFO

Keywords:

Industry 4.0

Predictive maintenance

Knowledge-based system

Chronicle mining

Ontology reasoning

ABSTRACT

In the context of Industry 4.0, smart factories use advanced sensing and data analytic technologies to understand and monitor the manufacturing processes. To enhance production efficiency and reliability, statistical Artificial Intelligence (AI) technologies such as machine learning and data mining are used to detect and predict potential anomalies within manufacturing processes. However, due to the heterogeneous nature of industrial data, sometimes the knowledge extracted from industrial data is presented in a complex structure. Therefore formal knowledge representation methods are required to facilitate the understanding and exploitation of the knowledge. Furthermore, as the Cyber-Physical Systems (CPS) are becoming more and more knowledge-intensive, uniform knowledge representation of physical resources and real-time reasoning capabilities for analytic tasks are needed to automate the decision-making processes for these systems. These requirements highlight the potential of using symbolic AI for predictive maintenance.

To automate and facilitate predictive analytics in Industry 4.0, in this paper, we present a novel Knowledge-based System for Predictive Maintenance in Industry 4.0 (KSPMI). KSPMI is developed based on a novel hybrid approach that leverages both statistical and symbolic AI technologies. The hybrid approach involves using statistical AI technologies such as machine learning and chronicle mining (a special type of sequential pattern mining approach) to extract machine degradation models from industrial data. On the other hand, symbolic AI technologies, especially domain ontologies and logic rules, will use the extracted chronicle patterns to query and reason on system input data with rich domain and contextual knowledge. This hybrid approach uses Semantic Web Rule Language (SWRL) rules generated from chronicle patterns together with domain ontologies to perform ontology reasoning, which enables the automatic detection of machinery anomalies and the prediction of future events' occurrence. KSPMI is evaluated and tested on both real-world and synthetic data sets.

1. Introduction

In today's manufacturing, increasing global competition, fast technology evolution and customers' perceptions of product quality have triggered the demand for future strategic plans and advanced manufacturing techniques. To meet these challenges, traditional manufacturing companies are leveraging smart and digital technologies to achieve higher productivity and increased automation. This leads to a new industrial stage where vertical and horizontal manufacturing process integration and product connectivity are helping companies to achieve higher industrial performance [16]. This digitalisation and automation process is called *Industry 4.0*. In Industry 4.0, automatic data exchange and data analysis open up opportunities for manufacturers to further optimise the production processes. Collecting data from various components of a production line and analysing them in a scalable Cloud infrastructure can significantly improve the productivity, reliability, and availability of production systems in heterogeneous environments [41]. With the current trend of automation and data exchange in manufacturing

technologies, traditional manufacturing factories are transforming into so-called "smart factories", which apply high-tech sensing and computation technologies on different manufacturing processes and production systems.

With the development of internet technologies, smart factories are benefiting from ubiquitous connectivity and seamless data exchange among manufacturing systems. This trend is a key enabler for Industry 4.0, where manufacturing entities are equipped with smart technologies such as Artificial Intelligence (AI), Internet of Things (IoT), Cyber-Physical Systems (CPS), and Cloud Computing. These advanced technologies are integrated to ensure more reliable, increased automation, and self-monitoring manufacturing systems.

In industry, a *predictive maintenance* task relies on the monitoring of a measurable system diagnostic parameter, which identifies the state of a system [25]. According to the current state of a machine, if any fault or failure exists, a diagnosis can be launched to determine the causes of the fault or failure. Also, based on the characteristics of the fault or failure, analysis about how they will propagate and evolve over time can be performed. In this way, machine or mechanical system deterioration tendency and the location of a failure can be predicted. Based on the prediction results, maintenance

*Corresponding author
ORCID(s):

decisions, such as calling the intervention of a machine operator are proposed according to the severity of anomalies, to prevent the halt of the production lines and to minimise economic loss.

Recently, manufacturing companies are integrating advanced AI techniques to construct CPSs, which are the central technique of Industry 4.0. As the data generated by sensors and networked machines gets higher in volume, a CPS is equipped with advanced technologies to obtain the capabilities of self-awareness and self-maintenance. These capabilities contribute significantly to the resilience, automation, and productivity of the CPS, which ensures the CPS to propose predictive decisions in an intelligent and optimal manner. Because of these benefits, CPS-based predictive maintenance has become a promising approach to detect and predict anomalies in manufacturing processes [41].

To deal with the challenge of big industrial data, manufacturers and researchers have devoted themselves to come up with advanced data management and analysis solutions. In terms of data management, statistical AI technologies are using new processing models to extract valuable information from heterogeneous data sources (e.g. product data, operational data, environment data). These technologies aim to achieve in-depth understanding and gain insight from data for accurate and timely decision making [69]. Also, jointly used with Cloud Computing and CPS, statistical AI technologies allow production systems to be self-aware and self-maintained. This ensures production systems to self-access their health conditions and the level of degradation, for them to perform smart maintenance decisions to avoid potential hazards and anomalies.

However, due to the heterogeneous nature of industrial data, sometimes the knowledge extracted from industrial data is presented in a complex structure. Therefore formal knowledge representation methods are required to facilitate the understanding and exploitation of the knowledge. Also, the task and context-specific nature of statistical AI methods have limited their reusability and implementation in knowledge-intensive manufacturing environments. This hampers the data-driven models to be reused under different contexts for automatic decision making. Furthermore, as the manufacturing domain is becoming more and more knowledge-intensive, uniform knowledge representation of physical resources and real-time reasoning capabilities for analytic tasks are needed to automate the decision-making processes for these systems. These limitations have highlighted the potential of using semantic technologies such as domain ontologies and rule-based systems for predictive maintenance.

To automate and facilitate predictive maintenance in the context of Industry 4.0, in this paper, we present a Knowledge-based System for Predictive Maintenance in Industry 4.0, named KSPMI. KSPMI is developed based on a novel hybrid approach that leverages both statistical and symbolic AI technologies. The execution of the software starts from the chronicle mining (a special type of sequential pattern mining approach) process on industrial data sets, after which machine degradation models are extracted from the data in-

dicating the time of occurrences of future machinery failures. After that, symbolic AI technologies, especially domain ontologies and Semantic Web Rule Language (SWRL) rule-based reasoning, use the extracted chronicle patterns to query and reason on system input data with rich domain and contextual knowledge. In this way, KSPMI enables the detection of future machinery failures as well as the prediction of their time of occurrence. The prediction of failures is associated with a pair of numeric values, which give the upper and lower numeric bounds of their time of occurrence. To improve the performance of failure prediction, KSPMI implements a rule pruning process for selecting a subset of best-quality SWRL rules for ontology reasoning. The rule pruning process is followed by an expert rule integration process, which enables the rule base to improve its quality progressively. During the integration process, KSPMI is programmed able to detect and avoid potential rule quality issues such as rule conflict, subsumption, and redundancy. The performance of KSPMI is evaluated and tested on both real-world and synthetic data sets. To the best of our knowledge, KSPMI is the first software that combines chronicle mining and ontology reasoning for Industry 4.0 predictive maintenance.

The remainder of this paper is structured as follows. Section 2 gives a comprehensive literature review on the existing approaches for Industry 4.0 predictive maintenance. Section 3 introduces the theoretical foundations of this paper. Within this section, formal definitions of relevant concepts and notions are given. Section 4 demonstrates the proposed novel hybrid approach that leverages both statistical and symbolic AI technologies for predictive maintenance. The proposed approach gives the foundation for the development work of KSPMI. Section 5 shows the implementation of KSPMI on two types of data sets. The first experimentation is performed on a real-world data set collected from a semi-conductor manufacturing process. The second experimentation is conducted on synthetic data sets. Section 6 concludes the paper and gives future perspectives.

2. Related work

Over the last decades, a considerable amount of research efforts have been undertaken to address the development of different models and systems for industrial predictive maintenance. In the context of a Industry 4.0 predictive maintenance task, appropriate method selection is vital to manufactures. A proper selection of an AI-based model or approach requires not only a thorough mathematical understanding of it but also the knowledge of how to implement it in real-world scenarios [60].

In this section, we classify the existing approaches and systems for industrial predictive maintenance into four categories: i) the knowledge-based systems; ii) the physical model-based approaches; iii) the data-driven approaches; iv) the hybrid model-based approaches. As KSPMI is developed based on knowledge-based and data-driven approaches, the literature review focuses mainly on these two types of

technologies.

2.1. Knowledge-based systems

A knowledge-based system maintains a knowledge base that stores the symbols of a computational model in the form of statements about the domain and performs reasoning by manipulating these symbols [26]. This type of systems uses explicit representations of knowledge in the form of words and symbols, which eases the understanding of the knowledge by a human or a computer. Also, they help to solve the semantic gap issue and ensure semantic interoperability among different systems and users. Normally, this type of system consists of a knowledge base that contains accumulated knowledge acquired from domain experts and a rule base for applying acquired knowledge to particular problems. A rule base contains a set of IF-THEN-based logical rules, which are formulated based on heuristic facts acquired from previous system behaviour or domain experts. An inference engine is another key component of the system that applies logical rules to the knowledge base to deduce new information.

One of the earliest knowledge-based systems for predictive maintenance was proposed in [7]. To enable failure detection, an expert system called FDPM was developed to detect incipient disturbances, locate the source of incipient behaviour, and classify different types of disturbance. The main components of FDPM include an expert system engine, a knowledge base, mathematical and neural network models of ageing of distribution equipment, and historical measurements databases [7]. To integrate reliable information with effective decision support for asset management, an expert system was developed to perform fuzzy logic reasoning for decision making [21]. In their work, a traditional expert system was fused with a fuzzy logic scheme. When dealing with a conflict situation or the formulation of expert knowledge, fuzzy concepts were proposed to describe vague concepts regarding asset management. In [6], a framework of decision support system for civil engineering was proposed. The framework equipped traditional decision support system with symbolic reasoning and explanation capabilities. Because of this, the system was able to integrate real-world information with a knowledge management module that models the structure health of buildings.

As an explicit specification of a conceptualisation for a domain of interest, ontologies form a fundamental component of a knowledge-based system. In the manufacturing domain, ontologies play a key role in many distributed intelligent systems as they provide a shared, machine-understandable vocabulary for information exchange among dispersed agents [11, 10, 8, 24, 22]. To model manufacturing products, in [48], the authors developed ONTO-PDM, which is an ontological model considered as a facilitator for interoperating all application software that shared information during the physical product lifecycle. The distinctive merit of this ontology was its incorporation of standardisation initiatives, such as the International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC)

standards. Another domain ontology named Manufacturing Predictive Maintenance Ontology (MPMO) was developed in a recent research paper [9]. MPMO was developed to formalise essential concepts and relations about condition monitoring and was used within a Cyber-Physical System to enable real-time predictive maintenance. To enable anomaly prediction, MPMO was used together with SWRL rules to enable ontology reasoning for detecting and predicting possible machinery failures that may happen within manufacturing processes. The effectiveness and usefulness of MPMO were tested on a real-world data set collected from a semiconductor manufacturing process. Another example of an ontology is the Sensing System Ontology, described in [44]. This ontology was developed to define the embedded sensing systems for industrial Product-Service Systems (PSSs). It was used as the backbone of the PSS knowledge-based framework, and it described the sensors that were embedded in PSSs for the aim of providing customised services for users. The ontology was tested on an industrial use case whose objective is to perform health monitoring on laser cutting machines. In this use case, the Sensing System Ontology is used to support the knowledge repository of a collaborative condition monitoring system named ICP4Life platform [44].

Normally, for the classic logic used in a knowledge-based system, a statement can be either true or false, which means a piece of data is classified either inside or outside of a set. However, to solve a real-world problem, sometimes it is not necessary to define a membership with such precision. In this context, fuzzy systems are developed, within which the IF-THEN-based logical rules are intentionally made imprecise. This type of systems use fuzzy set theory to overcome the deficiency of traditional expert systems by assigning partial set membership to data based on their “degree of truth” [60]. In this way, input data can belong to a fuzzy variable with a certain degree, allowing it to be part of more than one fuzzy variable at the same time. This fuzzification approach assures a better description and expression of imprecise knowledge [4].

2.2. Physical model-based systems

Physical models use explicit mathematical representation to formalise the physical understanding of a degrading machine or equipment [51]. This type of models address a predictive maintenance task by solving a deterministic equation, or a set of equations derived from extensive empirical data [60]. By using physical models, predictive maintenance of a physical asset is achieved through the interpretation of the acquired knowledge of a manufacturing process and through the analysis of possible hazards that may cause a failure. Normally, the main steps of a physical model-based predictive maintenance approach include failure modes and effects analysis (FMEA), feature extraction, and remaining useful life (RUL) estimation [50].

The advantages of physical models are their capability of direct incorporation of existing physical mechanisms that have been proved and well-understood by extensive and ex-

haustive empirical testing [60]. As the physical understanding of a monitored system improves, a physical model can update itself to improve its accuracy and quality. Furthermore, changes in the model outputs are described by the residuals (the differences between the reality and the model), which normally have a direct and translatable physical meaning. This eases the interpretation of the outputs of a physical model [60]. Because of these advantages, physical models have been widely used in the predictive maintenance of different physical assets, such as air vehicles [55], turbine engines [43, 60], and military systems [20], etc.

The main disadvantage of a physical model is the difficulty of assigning appropriate parameters used in the model. Since the massive and multivariate data required for the assignment of parameters are usually not available, it is hard to quantitatively characterize the system behavior [60]. Also, since fault and failure mechanisms may vary from one equipment to another, this type of models are normally equipment-dependent. Thus it is hard to identify the fault and failure mechanisms of a newly monitored machine component/machine without interrupting system operation [31].

2.3. Data-driven approaches

In recent years, rapid advances have been made in the research of data-driven models and approaches for accurate predictive maintenance. These models perform like a black box that learn the behaviour of physical assets directly from their operation data [35]. Within a data-driven approach, knowledge about machines is extracted internally from machine operation data instead of externally from domain experts. Normally, data-driven approaches are classified into machine learning (ML) techniques, and statistical techniques [19, 52].

Over the past decades, ML approaches have been proven to be effective solutions for predictive maintenance. The implementation of ML models have been facilitated by the growing capabilities of hardware, cloud computing, and newly introduced state-of-the-art algorithms [62].

Depends on the characteristics of data, the learning process of ML-based predictive maintenance can be categorized into three types [35]:

- Supervised learning which is applied to labeled data. This type of methods aim to learn a function that maps an input to an output based on a set of labeled training examples. Typical learning algorithms for supervised learning are Support Vector Machine (SVM) [40], Artificial Neural Networks (ANN) [32, 34], Logistic Regression [23], Naive Bayes [13], Random Forests [13], and Decision Trees [37], etc.
- Unsupervised learning applied to unlabeled data. In unsupervised learning, the training data is not associated with any corresponding target values. In other words, no labels are given to the learning algorithm, leaving it on its own to discover patterns and structure from the input data. The objective of an unsupervised learning process is to segregate data points with similar traits and assign them into clusters, which is known

as *clustering*, or to construct an estimate of the distribution of the input data, which is known as *density estimation*. Common unsupervised learning methods include k-means Clustering [1], Gaussian Mixture Models [54], Self-organizing Maps [64], etc.

- Semi-supervised learning applied to both labelled and unlabeled data. This type of learning methods is a learning paradigm concerned with the study of how computers and natural systems such as humans learn in the presence of both labelled and unlabeled data [68]. In addition to unlabeled data, semi-supervised learning algorithms are provided with a certain level of supervision, but not for all the training examples. Semi-supervised learning requires less human effort than supervised learning, meanwhile giving higher accuracy than unsupervised learning [68]. Because of these advantages, it has been pervasively implemented in real-world practices. Typical techniques used in this field are Self-Training algorithms [70], Generative Models [38], Semi-supervised Support Vector Machines [42], Graph-Based Algorithms [66], and Multi-view Algorithms [47], etc.

Facilitated by the capabilities of the advanced techniques and state-of-the-art algorithms, ML-based approaches have shown to be promising and effective solutions for industrial predictive maintenance. The algorithms and techniques introduced in this subsection have been widely used in real-world practices to reduce the maintenance cost and production downtime [62].

2.4. Statistical learning-based approaches

Within this kind of approach, predictive maintenance is achieved by fitting the empirical model (a function) as close as possible to the collected data and extrapolating the fitted curve to failure criteria [35]. A typical statistical model is a regression method for trend extrapolation, which is based on linear, exponential, or logarithmic functions. The common methods used in statistical learning-based approaches are i) Stochastic filtering [36]; ii) Particle filters and their variants [29]; iii) Hidden Markov models [61]; iv) Time series analysis [49].

Because of the low cost of deployment and better applicability than physical models, data-driven models have been widely applied to industrial predictive maintenance. However, the main disadvantage of this type of model is their demand for higher volume data than physical models. To well train a data-driven model, sufficient run-to-failure data ("sufficient" quantity means that data have been observed for all fault modes of interest [63]) needs to be collected for the model to capture complex relations among data [35]. This means a large amount of machine historical data needs to be collected ahead of time for obtaining high-quality and accurate data-driven models.

2.5. Hybrid models

A hybrid model applies both physics-based and data-driven approaches. As mentioned in the previous subsection,

data-driven models are feasible to be used when the required big data is easy to collect. However, it is usually the case that only part of machine historical data can be obtained [65]. In this context, the data-driven approach is jointly used with the physics-based approach for effectively identifying machine conditions. Existing hybrid model-based predictive maintenance approaches can be classified into two types: i) series approaches and ii) parallel approaches.

In series approaches, physical models are combined with online parameter estimation techniques to update model parameters when new data are available [35]. Data-driven methods are used to tune the parameters of physical models. This type of approaches have been applied to the predictive maintenance of various equipments, such as power metal-oxide-semiconductor field-effect transistors (MOSFETs) [14], printed circuit card assemblies [51], and Lithium-ion (Li-ion) batteries [56], etc.

Within parallel approaches, data-driven models are trained to predict the residuals not explained by the first principle model [35]. Normally, the hybrid model used in a parallel approach is created with an individual approach, which is either physics-based or data-driven. Therefore, the accuracy of a parallel hybrid model is normally higher than a series hybrid model [35]. However, implementing a parallel hybrid model requires several steps, which leads to a higher modelling complexity than series approaches-based models. Because of this, parallel hybrid models consume more computational time than series hybrid models.

2.6. Summary

Predictive maintenance is a key technique implemented in smart factories to improve the availability, reliability, and productivity of manufacturing systems. In this section, we have reviewed the existing models and approaches for industrial predictive maintenance by classifying them into four categories: knowledge-based systems, physical model-based approaches, data-driven approaches, hybrid model-based approaches. The classification is followed by the demonstration of the advantages and disadvantages of the existing models.

We use Fig. 1 to summarise our classification of the common predictive maintenance approaches, where the existing approaches are structured into four levels, marked with different colours. We first mark the common approaches with a superclass (the rectangle with blue colour) and then further divide the superclass into four sub-classes (rectangles with green colour). After that, the more specific models are presented by the rectangles with yellow colour. At last, the specific models are further classified into domain-specific techniques, which are represented by the rectangles with grey colour.

Since the industrial domain is becoming more and more knowledge-intensive, it is always beneficial to incorporate domain and contextual knowledge into the production process. Also, strong reasoning capabilities are required for the predictive maintenance system to predict machinery failures and propose maintenance decisions automatically. These rea-

soning capabilities also include using reasoners to check logic inconsistency and reason about the context. These open challenges have highlighted the need to develop an intelligent predictive maintenance system to enable automatic anomaly detection and prediction. To achieve this goal, we pay special attention to knowledge-based models/systems. This type of models/systems emulates the performance of a human expert in a specific domain for solving real-world problems. In predictive maintenance, a knowledge-based model/system uses knowledge management and knowledge representation techniques (e.g. semantic technologies [18]) to promote the prediction and diagnosis of machinery failures and to propose intelligent maintenance strategies. On the other hand, we leverage the strong statistical data processing capability of data-driven approaches, especially pattern mining techniques. The data-driven approaches will equip KSPMI with the ability to extract useful and informative patterns out of industrial data. The extracted patterns reflect real-world situations and indicate the machine degradation trend that may potentially lead to machinery failures.

3. Theoretical foundations

In this section, we introduce the theoretical foundations and basic notions that are necessary for describing the functionalities of KSPMI. We start with the concepts relevant to chronicle mining. The goal of chronicle mining is to extract frequent sequential patterns to predict machinery failures and their time of occurrence.

3.1. Chronicle mining

Chronicle mining is a special type of sequential pattern mining approach that can derive the order and temporal information of events. The aim of this mining process is to extract sequential patterns that contain rich temporal information from data. We first give the foundations of chronicle mining. The following definitions and notions are given by the literature paper [58].

Definition 1 (Event). Assume $\mathbb{E}$ is a set of event types, and $\mathbb{T}$ is a time domain such that $\mathbb{T} \subseteq \mathbb{R}$. An event is a couple (e, t) where $e \in \mathbb{E}$ is the type of the event and $t \in \mathbb{T}$ is its time.

A set of events may appear together in a certain order to form a *sequence*.

Definition 2 (Sequence). Let $\mathbb{E}$ be a set of event types, and $\mathbb{T}$ a time domain such that $\mathbb{T} \subseteq \mathbb{R}$. $\mathbb{E}$ is assumed totally ordered and is denoted $\leq_{\mathbb{E}}$. According to the definition in [15], a sequence is a couple $\langle SID, \langle (e_1, t_1), (e_2, t_2), \dots, (e_n, t_n) \rangle \rangle$ such that $\langle (e_1, t_1), (e_2, t_2), \dots, (e_n, t_n) \rangle$ is a sequence of events. For all $i, j \in [1, n]$, $i < j \Rightarrow t_i \leq t_j$. If $t_i = t_j$ then $e_i <_{\mathbb{E}} e_j$.

As mentioned before, a *sequence* may contain a set of events that appear in a certain order. When these events are time-stamped, it allows us to calculate the quantitative time intervals among different events. The computation of

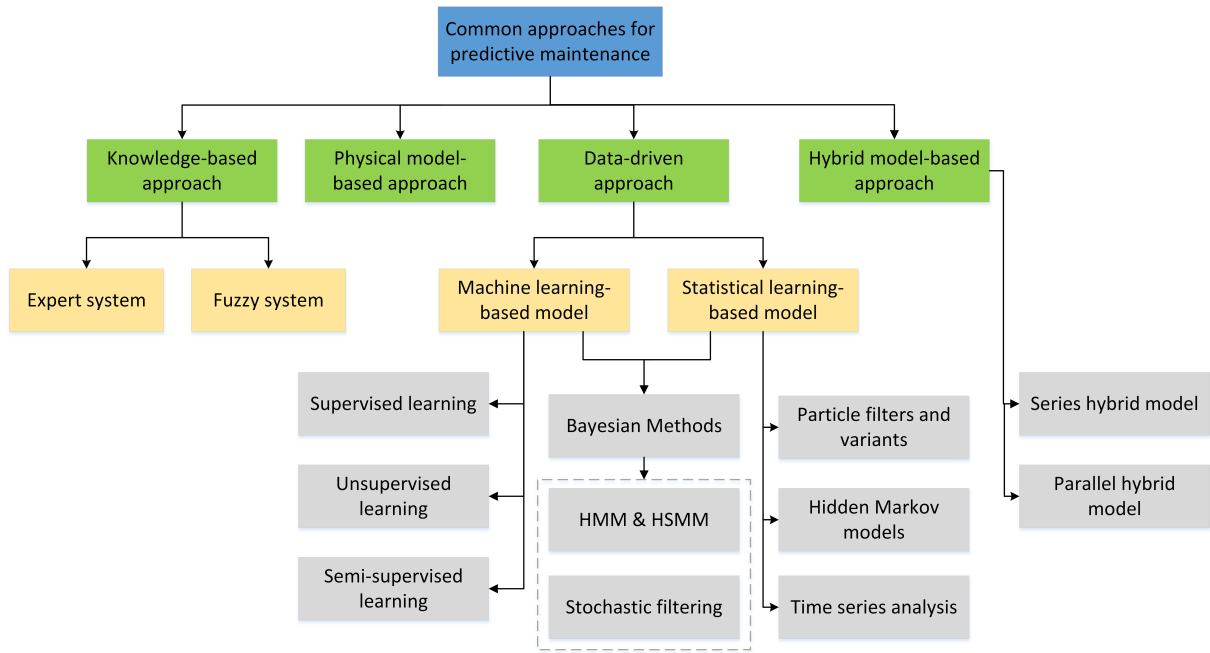


Figure 1: Classification of the common predictive maintenance approaches.

these quantitative time intervals is of paramount importance for predicting the time of occurrence of future failures. The time intervals together with their quantitative temporal values form the definition of *temporal constraints*.

Definition 3 (Temporal constraint). A temporal constraint is a quadruplet (e_1, e_2, t^-, t^+) , denoted $e_1[t^-, t^+]e_2$, where $e_1, e_2 \in \mathbb{E}$, $e_1 \leq_{\mathbb{E}} e_2$ and $t^-, t^+ \in \mathbb{T}$.

Within this definition, t^- and t^+ are two integers that stand for lower and upper boundaries of the associated time interval, such that $t^- \leq t^+$. A couple of events (e_1, t_1) and (e_2, t_2) are said to satisfy the temporal constraint $e_1[t^-, t^+]e_2$ iff $t_2 - t_1 \in [t^-, t^+]$.

We also define that $e_1[a, b]e_2 \subseteq e'_1[a', b']e'_2$ iff $[a, b] \subseteq [a', b']$, $e_1 = e'_1$, and $e_2 = e'_2$.

With introducing the definitions of *events* and *temporal constraints* within in a data sequence, the concept of *chronicle* is defined as follows [15].

Definition 4 (Chronicle). A chronicle is a pair $C = (\mathcal{E}, \mathcal{T})$ such that:

1. $\mathcal{E} = \{e_1 \dots e_n\}$, where $\forall i, e_i \in \mathcal{E}$ and $e_i \leq_{\mathbb{E}} e_{i+1}$.
2. $\mathcal{T} = \{t_{ij}\}_{1 \leq i < j \leq |\mathcal{E}|}$ is a set of temporal constraints on $\mathcal{E}$ such that for all pairs (i, j) satisfying $i < j$, t_{ij} is denoted by $e_i[t_{ij}^-, t_{ij}^+]e_j$.

Within the above definition, $\mathcal{E}$ is called the episode of C , according to the definition of episode's discovery in sequences [15].

Within a chronicle mining process, *chronicle support* is an important measure indicating the frequency of an extracted chronicle. We formalise the definition of *chronicle support* by the following definition.

Definition 5 (Chronicle support). An occurrence of a chronicle C in a sequence S is a set $(e_1, t_1) \dots (e_n, t_n)$ of events of the sequence S that satisfies all temporal constraints defined in C [57]. The support of a chronicle C in a sequential data set SD is the number of its occurrences in SD with maximum one occurrence in each data sequence S , or the percentage of its occurrences (with maximum one occurrence in each S) over all data sequences in SD .

To illustrate these aforementioned definitions, we give an example of demonstrating a data sequence and a chronicle. In the upper part of Fig. 2, we assume a data sequence S contains three events $\langle A, B, C \rangle$, represented as follows, time constraints that describe the pattern $\{A, B, C\}$ are noted by $A[2,5]B$, $B[1,5]C$ and $A[6,7]C$. Here $[2,5]$, $[1,4]$ and $[6,7]$ lower and upper boundaries of the time intervals among events.

After applying the chronicle mining approach on this data sequence, the generation of temporal constraints is enabled. As a result, these events are represented as a graphical way, as shown in the lower part of Fig. 2. In the figure, events are represented by the circle-shaped nodes, and temporal constraints are displayed as the edges among these three events. The values associated to each edge are quantitative numerical boundaries of temporal constraints. For each temporal constraint, a lower and upper numerical boundary are extracted.

In the domain of predictive maintenance, chronicle mining has been used to detect machine anomalies in advance [59, 57]. To enable the detection of anomalies, a special type of chronicles need to be introduced. Within this type of chronicles, the last event stands for a machinery failure, e.g. event C in Fig. 2. In this context, event A and B are

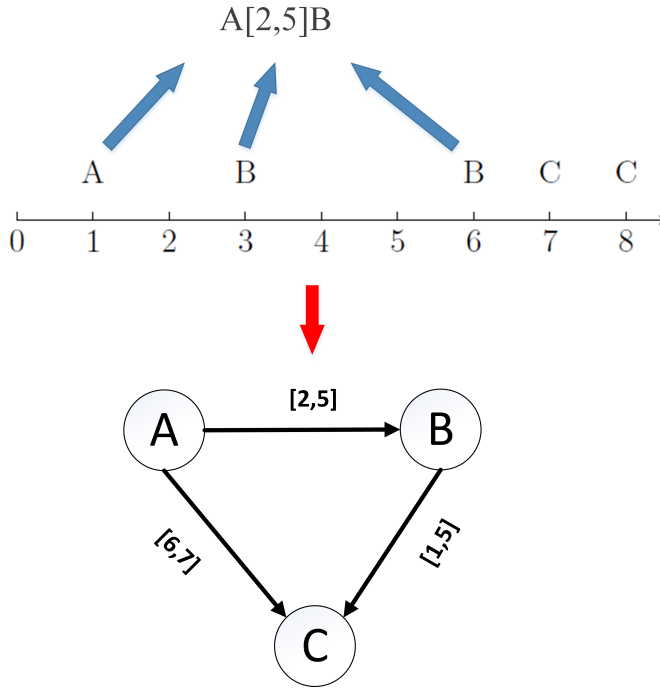


Figure 2: Example of a chronicle extracted from sequential data.

normal events that lead to a machinery failure. Based on this description, we introduce the concept of *failure chronicle*.

Definition 6 (Failure chronicle). For a chronicle $C_F = (\mathcal{E}, \mathcal{T})$, C_F is a failure chronicle if and only if the events that describe it are set according to their order of occurrence in the sequence, and within the chronicle there is a event that represents the failure, i.e. for $\mathcal{E} = \{e_1 \dots e_n | e_i \leq_{\mathcal{E}} e_{i+1}, i \in [1, n]\}$, e_n is the failure event.

For KSPMI, the output of the chronicle mining process is a set of frequent failure chronicles. These frequent failure chronicles contain information about the normal and abnormal events (failures) that are useful for predicting future events' occurrence.

3.2. Ontologies and SWRL rules

As KSPMI is a knowledge-based predictive maintenance system, it maintains a knowledge base that incorporates domain knowledge to operate. Normally, the domain knowledge is structured and represented by formal conceptual models, such as ontologies [27]. In computer science, an ontology is considered as “an explicit specification of a conceptualization for a domain of interest [28].” According to [5], the formal definition of an ontology is given as follows:

Definition 7 (Ontology). An ontology is a quadruplet $O : \langle C, P, Sub, Applic \rangle$, where:

- C is the set of the classes used to describe the concepts of a domain of interest.

- P is the set of properties used to describe the individuals of the C classes.
- Sub is the subsumption relation function defined as $Sub : C \rightarrow 2^C$, where 2^C denotes the power set of C . For a class c_i in the ontology, it associates its direct subsumed classes. Class C_1 subsumes class C_2 iff $\forall x \in C_2, x \in C_1$.
- $Applic$ is a function defined as $Applic : C \rightarrow 2^P$. This function associates to each ontology classes to those properties (object properties and data properties) that are applicable for each instance of this class.

Apart from domain ontologies, KSPMI also uses SWRL rules to perform ontology reasoning. Amenable to the Semantic Web standards, SWRL is a widely used standard language, proposed by the W3C community. SWRL is based on a combination of its sublanguages OWL DL and OWL Lite with the RuleMarkup Language. A SWRL rule is in the form of implication between an antecedent (body) and consequent (head), which can be interpreted in a way that whenever the conditions specified in the antecedent hold, then the conditions specified in the consequent must also hold [33]. In SWRL, a rule has the syntax: *Antecedent* $\rightarrow$ *Consequent*, where both the antecedent (body) and consequent (head) contains zero or more atoms. Atoms in SWRL rules can be the form of $C(x)$, $P(x, y)$, where $C(x)$ is an OWL class, P is an OWL property, and x, y are either variables, OWL individuals or OWL data values [33].

The reason we choose SWRL rules is two-fold. Firstly, SWRL provides model-theoretic semantics and has the advantage of its close association with OWL ontologies, which enables the definition of complex rules for reasoning about individuals in ontologies. Secondly, the use of SWRL to write rules is independent of rule implementation languages within rule engines, which has the advantage of the flexible selection of rule engines and inference platform [9].

In this work, we use SWRL rules to predict the machinery failures in manufacturing processes. These SWRL rules are generated from the frequent failure chronicles that are obtained from the chronicle mining process. We name them as *chronicle rules* and propose the formal definition of it:

Definition 8 (Chronicle rule). A chronicle rule is an implication $R : EC \wedge TEC \rightarrow TF$ such that:

- $EC = \{ec_1, \dots, ec_n\}$, which represents a set of non-failure events. The non-failure events in EC are ordered and denoted as $\leq_{EC}$. For $\forall i, ec_i \in EC$, and $ec_i \leq_{EC} ec_{i+1}$.
- $TEC = \{t_{ij} | 1 \leq i < j \leq |EC|\}$ is a set of temporal constraints on EC such that for all pairs (i, j) satisfying $i < j$, t_{ij} is denoted by $ec_i[t_{ij}^-, t_{ij}^+]ec_j$.
- $TF = \{t_{ik} | 1 \leq i < k \leq |EC| + 1\}$ is a set of temporal constraints among the non-failure events EC and a failure event F , such that for all pairs (i, k) satisfying $i < k$, t_{ik} is denoted by $ec_i[t_{ik}^-, t_{ik}^+]F$.

During the prediction process, to improve the performance of the system, we enable the system to prune a chronicle rule base for selecting a subset of best-quality rules. In Section 4, we introduce the two quality measures for rule pruning. Furthermore, KSPMI is programmed with capability to detect rule quality issues such as *conflict*, *subsumption*, and *redundancy*. In Section 3.3, we give formal definitions to these three rule quality issues.

3.3. Rule quality issues

Since the data set from where the chronicle rules are mined may not cover all real-world conditions, it is conceivable that the chronicle rules may fail to predict a failure. In this case, it is required to consult domain experts for decisions about failure prediction and maintenance. In this way, domain experts use their experience to assess the current state of the system and provide appropriate decisions, in the form of expert rules. By this, when there is a similar situation needs to be addressed, the rule which capitalises domain experts' experience will be launched together with the initial rules to identify potential failures and to make predictions.

However, when the expert rules are integrated into the initial rule base, a set of rule quality issues such as rule redundancy, conflict and subsumption may occur, thus reducing the reliability and performance of the rule-based reasoning process. Therefore, a rule base verification step is required to check any possible issues during the rule integration process. In this paper, *Redundancy*, *Conflict*, and *Subsumption* are considered as three important issues for the integration of the chronicle rules and the expert rules. Based on the definition of chronicle rules in Definition 8, the formal definition of these three issues are given as follows [12]:

Definition 9 (Chronicle rule redundancy). A chronicle rule $R : EC \wedge TEC \rightarrow TF$ and a rule $R' : EC' \wedge TEC' \rightarrow TF'$ are considered as redundant when the two rules have the same sets of events, and the events have the same temporal constraints, denoted as

$$ChroRedundancy(R, R') \iff (EC = EC') \wedge (TEC = TEC') \wedge (TF = TF'). \quad (1)$$

Definition 10 (Chronicle rule conflict). A chronicle rule $R : EC \wedge TEC \rightarrow TF$ and a rule $R' : EC' \wedge TEC' \rightarrow TF'$ are considered to have conflict when the two rules succeed in the same sets of non-failure events and the same temporal constraints for these non-failure events, but with conflicting temporal constraints of a failure, denoted as

$$ChroConflict(R, R') \iff (EC = EC') \wedge (TEC = TEC') \wedge (TF \neq TF'). \quad (2)$$

Definition 11 (Chronicle rule subsumption). A chronicle rule $R : EC \wedge TEC \rightarrow TF$ subsumes the chronicle rule $R' : EC' \wedge TEC' \rightarrow TF'$ when they have the same temporal constraints of a failure, but rule R contains additional restrictions on the set of non-failure events or on the temporal

constraints of these non-failure events. This subsumption relationship is denoted as

$$ChroSubsums(R, R') \iff (TF = TF') \wedge ((EC' \wedge TEC') \models (EC \wedge TEC)). \quad (3)$$

These definitions are used to detect issues with regard to rule base verification. When different rule bases are integrated, the aforementioned three rule quality issues are examined to refine the integrated rule base.

4. Proposed approach

This section gives the core approach based on which the KSPMI software is developed and programmed. As a hybrid predictive maintenance method that combines chronicle mining and semantic technologies, the proposed approach consists of two main phases. These two phases are further divided into four main processes: the frequent chronicle mining process, the SWRL rule generation and pruning process, the expert rule integration process, and the failure detection and prediction process. Fig. 3 gives an overview of our approach. We describe these four processes as well as their main steps below:

1. The frequent chronicle mining process. Chronicles are a type of sequential patterns that incorporates rich temporal information of different events for decision making. As introduced in Section 3.1, chronicles are essentially *event-time* pairs that represented in a graph format where vertices are events and edges are labelled with intervals representing the time between the two linked events [59]. Frequent chronicle mining is a special type of sequential pattern mining approach that extracts all frequent chronicles from data. The extracted chronicles contain information about the events' occurrence order as well as their time of occurrence. In this paper, we use the Clasp-CPM algorithm introduced in [58] to extract a set of frequent failure chronicles from industrial data.
2. The SWRL rule generation and pruning process. Within this step, a set of SWRL predictive rules are generated from the extracted frequent failure chronicles. These rules are IF-THEN-based logic statements that enable rule-based reasoning. To enable rule generation, we use the SWRL rule generation algorithm that was developed in our previous work [9]. The SWRL rule generation algorithm allows automatic transformation of the extracted chronicles into a set of SWRL predictive rules. The pseudo codes of this algorithm is shown in Algorithm 1. The algorithm first extracts different types of events from a failure chronicle, and then discover the order of these events as well as their temporal constraints. At last, a rule is generated as an implication between the antecedent and the consequent. After rule generation, a rule pruning process

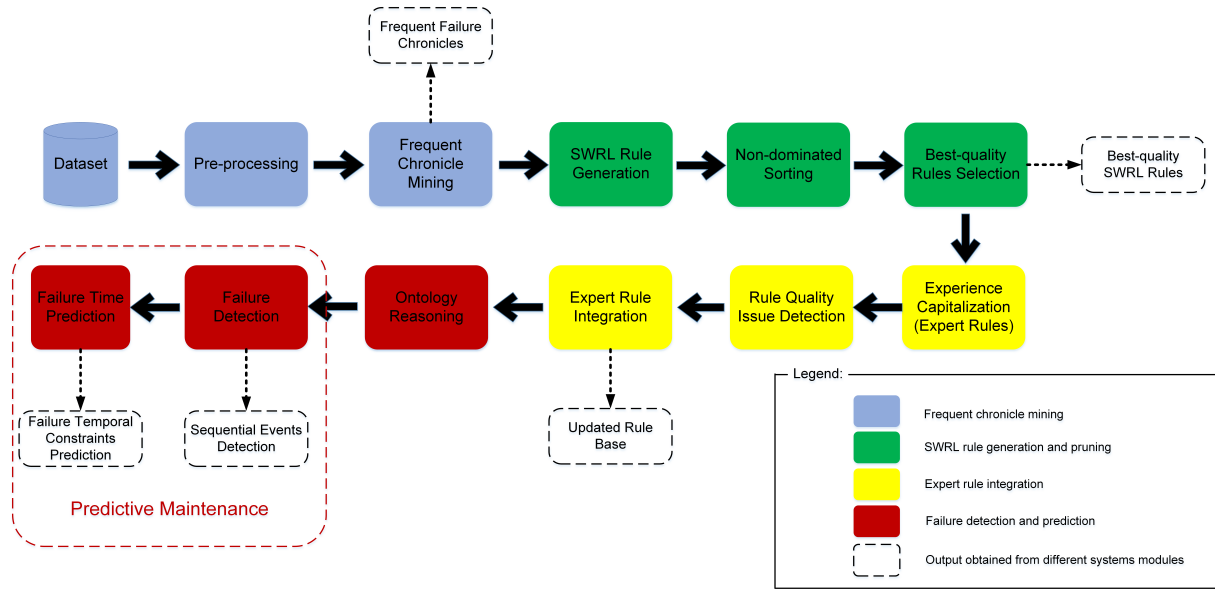


Figure 3: An overview of the proposed approach.

is evoked to remove low-quality rules. The rule pruning process is enabled by a multi-objective optimisation process that applied to rule *accuracy* and *coverage* measures. The definition of rule accuracy and coverage can be found in our previous paper [12]. This multi-objective optimisation step uses the fast non-dominated sorting algorithm introduced in [46] to select a subset of best quality rules based on the two quality measures. The best quality rules have the highest accuracy and coverage measures among the whole rule base thus are sorted within the first Pareto Front. After rule pruning, the best-quality rules construct a new chronicle rule base.

3. The expert rule integration process. Within this process, the experience of domain experts is capitalised in the form of expert rules. We aim to complement the chronicle rule base (rules generated from the previous step) with a set of expert rules. By this, we address the incompleteness issue of the chronicle rule base. During this integration step, a set of rule quality issues such as *redundancy*, *conflict*, and *subsumption* are examined when heterogeneous rules are combined to achieve a satisfactory level of reasoning performance. During the rule integration process, the automatic detection of rule quality measures is enabled by a novel algorithm proposed in our previous research [12]. The pseudo codes of this algorithm is shown in Algorithm 2. Within this algorithm, three functions extract different sets of atoms from an expert rule. After that, three steps are proposed to detect rule redundancy, subsumption, and conflict issues among the expert rule and the whole chronicle rule base. As a result, a refined rule based is obtained after integrating expert rules with the chronicle rule base.
4. The failure detection and prediction process. The de-

tection and prediction of machine anomalies are achieved by ontology reasoning. KSPMI uses the Manufacturing Predictive Maintenance Ontology (MPMO) as the domain ontology together with generated SWRL rules to perform reasoning. The formalism and description of MPMO can be found in our recently published paper [9]. As a result, the occurrence of machinery failures is detected. Also, the temporal information of the failures is predicted. By this, KSPMI is able to detect and predict possible machinery failures that may happen in the future.

KSPMI is developed into four system modules that correspond to these four processes. In the next section, we demonstrate the implementation of KSPMI on several real-world and synthetic data sets.

5. KSPMI: the knowledge-based predictive maintenance system

In Section 4, we introduced the four core modules of KSPMI. In this section, we first describe the development environment and programming tools for KSPMI^{1 2}. Then we introduce the core functionalities of KSPMI by following the key steps introduced in Section 4: frequent chronicle mining, SWRL rule generation and pruning, expert rule integration, failure detection and prediction.

5.1. Software development environment

During the development of KSPMI, several software and tools are used. They are listed as follows:

¹The source code and all used data sets for KSPMI can be found at: <https://github.com/caoppg/KSPMI.git>

²A demonstration video for KSPMI can be found at: <https://sites.google.com/view/kspmiknowledge-basedsystem/home>

Algorithm 1 Algorithm to transform a chronicle into a predictive SWRL rule [9].

Require: C_F : A chronicle within which the last event is a failure event, $\mathcal{E}$: the episode of C_F which contains different types of events in a chronicle.

Ensure: R

```

1:  $EL \leftarrow LastNonfailureEvent(C_F, \mathcal{E})$ 
2:  $\triangleright$  Extract the last non-failure event before the failure within a chronicle.
3:  $R \leftarrow \emptyset, A \leftarrow \emptyset, C \leftarrow \emptyset, Atom_a \leftarrow \emptyset, Atom_c \leftarrow \emptyset.$ 
4: for each  $e_i[t_{ij}^-, t_{ij}^+]e_j \in \mathcal{T}$  do
5:    $pe \leftarrow ProceedingEvent(e_i[t_{ij}^-, t_{ij}^+]e_j)$ 
6:    $\triangleright$  Extract the proceeding event of this time interval
7:    $se \leftarrow SubsequentEvent(e_i[t_{ij}^-, t_{ij}^+]e_j)$ 
8:    $\triangleright$  Extract the subsequent event of this time interval
9:    $Atom_a \leftarrow [t_{ij}^-, t_{ij}^+] \wedge pe \wedge se$ 
10:   $A \leftarrow Atom_a \wedge ([t_{ij}^-, t_{ij}^+] \wedge pe \wedge se)$ 
11: end for each
12: for each  $el \in EL$  do
13:    $ftc \leftarrow FailureTimeConstraint(el, TI)$ 
14:    $\triangleright$  Extract the time constraint between the last event before the failure and the failure event.
15:    $Atom_c \leftarrow el \wedge ftc$ 
16:    $C \leftarrow Atom_c \wedge (el \wedge ftc)$ 
17: end for each
18:  $R \leftarrow (A \rightarrow C)$ 
19: return  $R$ 

```

- Java Platform, Standard Edition (Java SE)³. Java SE is a computing platform for developing and deploying portable codes for desktop and server applications. Java is an object-oriented programming language designed for building up mobile applications, data processing tools, and embedded systems.
- Eclipse⁴. It is an integrated development environment (IDE) for computer programming. With its primary use for developing Java applications, Eclipse consists of a workspace and an extensible plug-in system for users to customize their development environment.
- Protégé⁵. Developed by Stanford University, Protégé is a free and open-source software for developing knowledge-based systems and editing ontologies. It can be extended by a set of plug-in applets and can be transferred into a Java-based Application Programming Interface (API) for the development of Java-based intelligent systems. During our development phase, three Protégé-based APIs have been used: OWL API⁶, SWRL API⁷, and SQWRL API⁸.

³<https://www.oracle.com/java/technologies/javase-downloads.html>

⁴<https://www.eclipse.org/>

⁵<https://protege.stanford.edu/>

⁶<http://owlapi.sourceforge.net/>

⁷<https://github.com/protegeproject/swrlapi>

⁸<https://github.com/protegeproject/swrlapi/wiki/SQWRL>

Algorithm 2 The algorithm to detect the three issues when an expert rule is integrated into the chronicle rule base [12].

Require:

- 1: $\mathbb{R}$: a chronicle rule base which contains a set of failure chronicles.
- 2: R_e : an expert rule which is in the form of a failure chronicle.

Ensure:

- 3: $\mathbb{R}'$: the integrated rule base.
- 4: $EE \leftarrow \emptyset, TEE \leftarrow \emptyset, TFE \leftarrow \emptyset, EC \leftarrow \emptyset, TEC \leftarrow \emptyset, TFC \leftarrow \emptyset, \mathbb{R}' \leftarrow \emptyset.$
- 5: $EE \leftarrow Non-failureEvent(R_e)$
- 6: $\triangleright$ Extract all the non-failure events of this expert rule.
- 7: $TEE \leftarrow Non-failureTemporalConstraints(R_e)$
- 8: $\triangleright$ Extract the temporal constraints of these non-failure events.
- 9: $TFE \leftarrow FailureTemporalConstraints(R_e)$
- 10: $\triangleright$ Extract the temporal constraints of the failure.
- 11: **for each** $R \in \mathbb{R}$ **do**
- 12: $EC \leftarrow Non-failureEvent(R)$
- 13: $\triangleright$ Extract all the non-failure events of this chronicle rule.
- 14: $TEC \leftarrow Non-failureTemporalConstraints(R)$
- 15: $\triangleright$ Extract the temporal constraints of the non-failure events in the chronicle rule.
- 16: $TFC \leftarrow FailureTemporalConstraints(R)$
- 17: $\triangleright$ Extract the temporal constraints of the failure in the chronicle rule.
- 18: **if** $EE = EC, TEE = TEC, TFE = TEC$, **then**
- 19: $ChroRedundancy(R_e, R)$ $\triangleright$ Rule redundancy issue is detected.
- 20: $Remove(R_e, \mathbb{R})$ $\triangleright$ Remove the redundant expert rule.
- 21: **else if** $TFE = TFC, EC = EE, TEC \subseteq TE$, **then**
- 22: $ChroSubsumes(R_e, R)$ $\triangleright$ Rule subsumption issue is detected.
- 23: $Remove(R, \mathbb{R})$ $\triangleright$ Remove the subsumed chronicle rule from the rule base.
- 24: $Integrate(R_e, \mathbb{R})$ $\triangleright$ Integrate the expert rule into the rule base.
- 25: **else if** $EE = EC, TE \subseteq TEC, TFE \neq TFC$, **then**
- 26: $ChroConflict(R_e, R)$ $\triangleright$ Rule conflict issue is detected.
- 27: $Remove(R, \mathbb{R})$ $\triangleright$ Remove the conflict chronicle rule from the rule base.
- 28: $Integrate(R_e, \mathbb{R})$ $\triangleright$ Integrate the expert rule into the rule base.
- 29: **else**
- 30: $Integrate(R_e, \mathbb{R})$
- 31: **end if**
- 32: **end for each**
- 33: $\mathbb{R}' \leftarrow \mathbb{R}$
- 34: **return** $\mathbb{R}'$

- Drools⁹. It is a business rule engine that provides forward and backward chaining inference capabilities. Drools have been widely used for developing production rule systems. In KSPMI, Drools is used to execute SWRL rules for ontology reasoning, which aims to predict the time of machinery failures.
- SPMF¹⁰. SPMF is an open-source data mining library written in Java, and it provides a set of pattern mining algorithms to discover patterns out of data. The main advantage of using SPMF is the source code of each algorithm in it can be easily integrated with other programs that are written in Java.

In the next sections, we demonstrate the key functionalities and main Graphical User Interfaces (GUI) of KSPMI. To do this, we conduct experimentation on both real-world and synthetic data sets.

5.2. Experimentation on a real-world industrial data set

KSPMI takes sequential data sets as input. Our first experimentation is conducted on the UCI SECOM data set [45], which contains measurements of features of semi-conductor production within a semi-conductor manufacturing process.

5.2.1. The UCI SECOM data set

In the UCI SECOM data set, 1567 data records and 590 attributes are collected. The rows in the data set stand for test points. Every test point consists of 589 attributes, where each attribute is associated with its numerical value. The values in the second last column are the timestamps when these test points are recorded. Since the timestamps are in ascending order, the test points are considered as a sequence of events that are timely ordered. A binary label is given in the last column for each row, which indicates a pass (0) or fail (1) of the recorded test point. In total, 104 number of records represent the failures of production. The data is stored in a raw text file, within which each line represents an individual example of recording with its timestamp. The features are separated by spaces. In the input data set, events are associated with timestamps, and each test point is given a binary label (e.g. -1 corresponds to normal status, and 1 corresponds to a failed status). Fig. 4 shows an excerpt of the UCI SECOM data set as system input.

However, some of the data contained in UCI SECOM data set is irrelevant to the failure prediction task thus is considered as noise. Also, due to the inter-dependency among individual features and the complex behavior of combined features, it is difficult to extract frequent chronicles and rules based on the analysis of all the 590 attributes. To address this challenge, feature selection methods [30] are used to identify and select the most relevant attributes in predicting the failures. The selected attributes are subsequently used to extract the key factors and patterns that lead to machine

failures. This reduces the data processing time and memory consumption.

After the feature selection step, data discretisation [53] is employed to discretise continuous values for obtaining nominal ones. Thereafter, to enable the frequent chronicle mining process, data sequentialisation is used to transform the data into the form of pairs (event, time stamp). As a result, data are organised in a sequential format, and each data sequence ends up with a failure. With obtaining data sequences that contain failures, the CloSpan algorithm [67] is applied to the pre-processed data set to extract frequent sequential patterns. After that, the Clasp-CPM algorithm introduced in [58] is used to extract the temporal constraints among these sequential patterns. Up to this step, we are able to obtain frequent failure chronicles that will be transformed into predictive SWRL rules.

5.2.2. The frequent chronicle mining GUI

The running of KSPMI starts with the frequent chronicle mining phase on input data. Within this phase, the frequent chronicle mining approach introduced in [57] is used to extract failure chronicles out of data. These failure chronicles describe a set of ordered events and the temporal constraints of failures. Fig. 5 shows the chronicle mining GUI of KSPMI.

In the chronicle mining interface, a set of extracted frequent failure chronicles are displayed in the table within Fig. 5. There are three important measures associated with each failure chronicle: *support*, *accuracy*, and *coverage*. Among them, the support value is calculated based on Definition 5. The computation of rule accuracy and coverage will be introduced in Section 5.2.3. With clicking on each table entry, the values of these three measures are shown on the right side. At the same time, the graph structure of the selected failure chronicle is also displayed. Within each graph, the event with integer 0 indicates a failure. The integers in other events are the nominal attributes we obtain after data discretisation.

At the bottom of the frequent chronicle mining GUI, users can click the “Transform the extracted chronicles into SWRL rules” button to proceed to the SWRL rule generation and pruning phase. After clicking this button, Algorithm 2, which is introduced in Section 4, is activated to generate a set of predictive SWRL rules from the extracted frequent failure chronicles. These SWRL rules construct a chronicle rule base. The constructed chronicle rule base will be pruned and then complemented by expert rules and then launched for failure prediction.

5.2.3. The SWRL rules generation and rule pruning GUI

After the frequent chronicle mining step, the extracted failure chronicles are transformed into a set of SWRL rules. Within this interface, users can first display the transformed rule base by clicking “Display”. Then, the syntax of rules is presented in the text area on the left side. Fig. 6 shows this GUI.

⁹<https://www.drools.org/>

¹⁰<https://www.philippe-fournier-viger.com/spmf/>

	A	B	C	D	E	F	G
1	0	1	2	3	4	5	6
2	3030.93	2564	2187.733	1411.127	1.3602	100	97.6133
3	3095.78	2465.14	2230.422	1463.661	0.8294	100	102.3433
4	2932.61	2559.94	2186.411	1698.017	1.5102	100	95.4878
5	2988.72	2479.9	2199.033	909.7926	1.3204	100	104.2367
6	3032.24	2502.87	2233.367	1326.52	1.5334	100	100.3967
7	2946.25	2432.84	2233.367	1326.52	1.5334	100	100.3967
8	3030.27	2430.12	2230.422	1463.661	0.8294	100	102.3433
9	3058.88	2690.15	2248.9	1004.469	0.7884	100	106.24
10	2967.68	2600.47	2248.9	1004.469	0.7884	100	106.24
11	3016.11	2428.37	2248.9	1004.469	0.7884	100	106.24
12	2994.05	2548.21	2195.122	1046.147	1.3204	100	103.34
13	2928.84	2479.4	2196.211	1605.758	0.9959	100	97.9156
14	2920.07	2507.4	2195.122	1046.147	1.3204	100	103.34
15	3051.44	2529.27	2184.433	877.6266	1.4668	100	107.8711
16	2963.97	2629.48	2224.622	947.7739	1.2924	100	104.8489
17	2988.31	2546.26	2224.622	947.7739	1.2924	100	104.8489
18	3028.02	2560.87	2270.256	1258.456	1.395	100	104.8078
19	3032.73	2517.79	2270.256	1258.456	1.395	100	104.8078
20	3040.34	2511.16	2207.389	962.5317	1.2043	100	104.0311
21	2988.3	2519.05	2208.856	1157.722	1.5509	100	107.8022
22	2987.32	2528.81					
23		2481.85	2207.389	962.5317	1.2043	100	104.0311

VN	VO	VP	VQ	VR	VS	VT
585	586	587	588	589	timestamp	fail
2.363					1216468500	0
4.4447	0.0096	0.0201	0.006	208.2045	1216470720	0
3.1745	0.0584	0.0484	0.0148	82.8602	1216473420	1
2.0544	0.0202	0.0149	0.0044	73.8432	1216478580	0
99.3032	0.0202	0.0149	0.0044	73.8432	1216480920	0
3.8276	0.0342	0.0151	0.0052	44.0077	1216489980	0
2.8515	0.0342	0.0151	0.0052	44.0077	1216496640	0
2.1261	0.0204	0.0194	0.0063	95.031	1216496700	0
3.4456	0.0111	0.0124	0.0045	111.6525	1216499040	0
3.0687	0.0212	0.0191	0.0073	90.2294	1216503300	0
3.2115	0.0355	0.0205	0.0071	57.8122	1216504620	1
8.5646	0.037	0.0279	0.0081	75.5077	1216507920	1
4.0926	0.0188	0.0098	0.0034	52.2039	1216524900	0
3.0063	0.0188	0.0098	0.0034	52.2039	1216628460	0
1.8483	0.0202	0.0289	0.0084	142.908	1216641180	1
1.5352	0.0174	0.0174	0.0045	100.2745	1216684980	0
2.1574	0.0184	0.0151	0.0042	82.0989	1216695540	0
2.0979	0.0184	0.0151	0.0042	82.0989	1216716060	0
2.3737	0.0184	0.0151	0.0042	82.0989	1216727220	0
3.3514	0.0229	0.0108	0.0032	47.1586	1216735200	0
2.3308	0.0229	0.0108	0.0032	47.1586	1216740600	0
2.7729	0.0175	0.006	0.0023	34.4153	1216790100	0

Figure 4: An excerpt of an input data set for KSPMI.



Figure 5: The chronicle mining GUI of KSPMI.

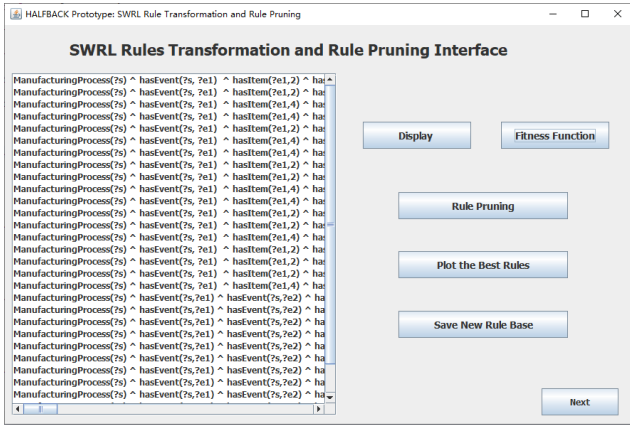


Figure 6: The SWRL rules generation and rule pruning GUI.

Normally, the number of chronicle rules transformed from chronicles is large. Due to a certain degree of imprecision in real-world data, some of the extracted chronicle rules may suffer from low-quality. As a result, low-quality chronicle rules (e.g. rules with low accuracy in failure prediction) may

reduce the efficiency and accuracy of rule-based reasoning. Therefore, a rule base pruning phase for obtaining a high-quality subset of rules is needed.

In this context, KSPMI implements a rule pruning process applied to the rule base obtained from frequent chronicle mining. To reduce the number of extracted rules and to achieve the best quality of the rule base, a multi-objective optimisation approach is applied. As introduced in Section 4, to evaluate the quality of the SWRL rules, two measures are computed. We identify accuracy and coverage as the two rule measures for evaluating the quality of SWRL rules, as these two measures are the important indicators of a rule's reliability [2]. These two measures can be derived based on the analysis of a 2×2 contingency table. A contingency table for a rule $R : A \rightarrow F$ is computed as follows:

Table 1

The contingency table for computing rule quality measures.

	F is true	F is not true	
A is true	n_{af}	$n_{a\bar{f}}$	n_a
A is not true	$n_{\bar{a}f}$	$n_{\bar{a}\bar{f}}$	$n_{\bar{a}}$
	n_f	$n_{\bar{f}}$	N

In Table 1, n_{af} denotes the number of training examples for which both the antecedent and the consequent of the rule are true. $n_{a\bar{f}}$ denotes the number of training examples for which the antecedent of the rule is true, and the consequent of the rule is not true. On the other hand, $n_{\bar{a}f}$ denotes the number of training examples for which the antecedent is not true, and the consequent of the rule is true. $n_{\bar{a}\bar{f}}$ denotes the number of training examples for which neither the antecedent and the consequent of the rule is true. n_a , $n_{\bar{a}}$, n_f , $n_{\bar{f}}$ are marginal totals, and N is the total number of training examples.

Based on the information provided by the contingency table, two basic evaluation measures can be computed: Ac -

accuracy and Coverage. Among them, the rule accuracy is defined as

$$Accuracy(R) = n_{af}/n_a, \quad (4)$$

and rule coverage is defined as

$$Coverage(R) = n_{af}/n_f. \quad (5)$$

The above two equations are used to compute the values of accuracy and coverage for the SWRL rules. Table 2 presents the average values of these two measures under different chronicle support. It can be observed from the table that as the chronicle support increases, the accuracy of rules also increases. It is reasonable as the support of extracted chronicles increases, more relevant chronicles are extracted. On the other hand, as the number of extracted rules decreases, the sequences that are covered by the rules decreases. This is the reason why the average value of coverage shows a downtrend.

Table 2
Two rule quality measures under different chronicle support.

Chronicle support	Accuracy	Coverage
0.5	76.52%	74.26%
0.6	74.14 %	75.71%
0.7	76.98 %	74.35%
0.8	79.33%	70.49%
0.9	82.56%	68.10%
1	84.45%	67.71%

The SWRL rules transformation and rule pruning GUI allows users to visualise the transformed rules and to select the best-quality rules by using a multi-objective optimisation approach. To achieve this goal, the fast non-dominated sorting approach introduced in [46] is used to obtain the ranking of Pareto dominance for chronicle rules. After that, the rules within the first front are selected to perform rule-based reasoning. In this way, a set of chronicle rules that have the best quality are launched for failure prediction.

After that, the fitness function of the multi-objective optimisation approach can be shown by clicking the “Fitness Function” button. The fitness function we use in this phase is the product of rule accuracy and coverage, as introduced by Equation 6. The fitness value reflects the quality of a rule. The higher this value is, the better quality of the SWRL rule is.

$$Fitness(R) = Accuracy(R) \times Coverage(R). \quad (6)$$

The button “Rule Pruning” launches the pruning of the rule base by using the fast non-dominated sorting algorithm on the two rule quality measures. After the pruning phase, a subset of best-quality rules are displayed on the left side, and the objective function values of them can be plotted by

clicking the button “Plot the Best Rules”. Fig. 7 shows the rule pruning results obtained after clicking the “Plot the Best Rules” button. Within the figure, the rules within the first Pareto Front are considered best quality. The best quality rules are then selected to form a chronicle rule base.

Up to this step, we have completed the SWRL rule generation and pruning step.

5.2.4. The expert rule integration GUI

After obtaining a set of best-quality rules, users can proceed to the expert rule integration phase, where expert rules are used to complement the chronicle rule base. The goal of integrating expert rules into the system is to improve the overall fitness of the whole rule base. To enable this step, a rule integration GUI is developed to allow users to interact with the expert rule base.

Fig. 8 shows the expert rule integration GUI of KSPMI. Users can first open the expert rule file, which is stored separately from the chronicle rule base. At the lower part of this GUI, potential rule quality issues are detected and then printed. Fig. 9 shows an example of an expert rule file. This rule file can be opened by clicking the button “Open Expert Rules”. The expert rules are the same format as the chronicle rules, with differences in rule atoms within either antecedent or consequent part.

The expert rules can be integrated into the chronicle rule base by clicking the “Input Expert Rule” button. After clicking this button, the system tries to push the input expert rule into the chronicle rule base. If there is a quality issue detected, appropriate actions are automatically performed according to the decision making process in Algorithm 2. Within this example shown in Fig. 8, a conflict issue is detected among the expert rule and chronicle rule base. The system then automatically removes the conflict chronicle rule, and add the expert rule into the rule base. The reason for this action is based on the assumption that we assign the fitness values of all expert rules to 1, which means the expert rules are always more reliable than chronicle rules.

On the other hand, if no issue detected, the input expert rule is directly integrated into the chronicle rule base. As a result, the integrated rule base is progressively updated with more expert rules integrated. These actions are also printed in the text area within the expert rule integration GUI.

5.2.5. The failure detection and prediction GUI

After the expert rule integration process, we proceed to the failure prediction step. The failure prediction work is achieved by using the Drools rule engine to perform ontology reasoning on the individuals that are populated in the MPMO ontology (this ontology is introduced in our previous paper [9]). After prediction, a SQWRL query is created to retrieve the prediction results. The SQWRL language takes an antecedent of a SWRL rule and effectively treats it as a pattern specification for a query. To extract the results, a SQWRL query replaces a rule consequent with a retrieval specification.

Fig. 10 shows the first round of prediction results. In this

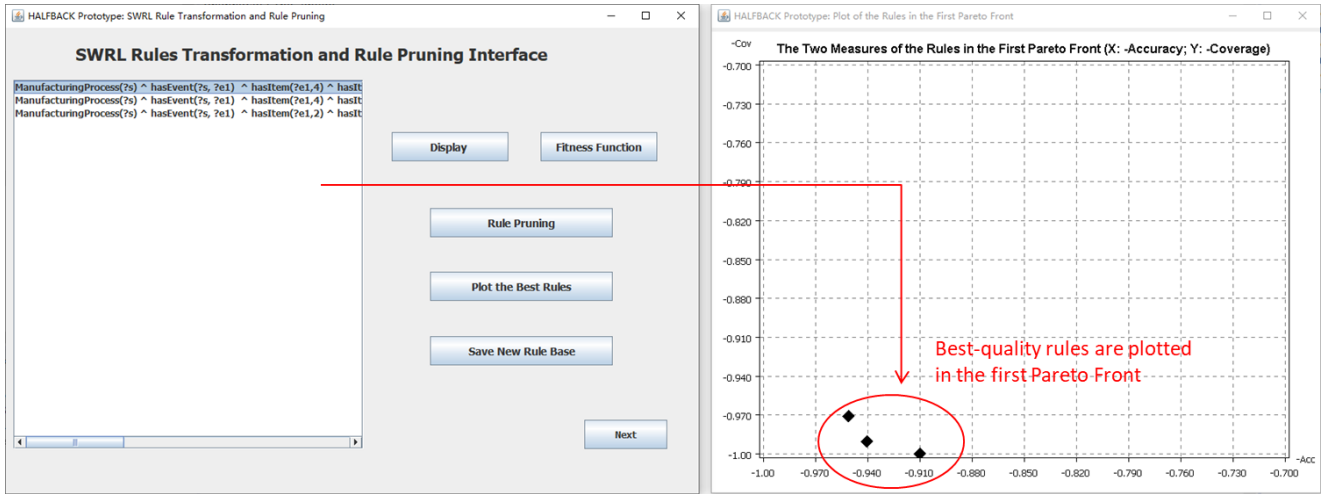


Figure 7: The rule pruning results. The best-quality rules are plotted within the first Pareto Front.

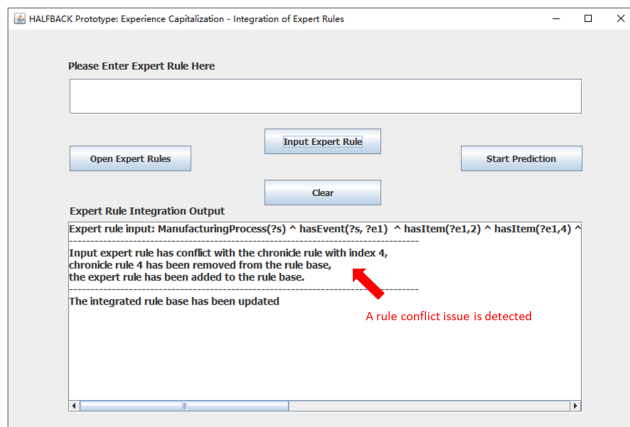


Figure 8: The expert rule integration GUI.

```

Expert rule 1:
ManufacturingProcess(?s) ^ hasEvent(?s, ?e1) ^ hasItem(?e1,2) ^ hasItem(?e1,4) ^ hasItem(?e1,5) ^ hasItem(?e1,7) ^
hasItem(?e1,11) ^ hasItem(?e1,17) ^ hasItem(?e1,19) -> hasMinF(?e1, 1) ^ hasMaxF(?e1, 2656921)

Expert rule 2:
ManufacturingProcess(?s) ^ hasEvent(?s, ?e1) ^ hasItem(?e1,2) ^ hasItem(?e1,4) ^ hasItem(?e1,5) ^ hasItem(?e1,7) ^
hasItem(?e1,11) ^ hasItem(?e1,17) ^ hasItem(?e1,19) -> hasMinF(?e1, 1) ^ hasMaxF(?e1, 2656921)

Expert rule 3:
ManufacturingProcess(?s) ^ hasEvent(?s, ?e1) ^ hasItem(?e1,4) ^ hasItem(?e1,5) ^ hasItem(?e1,7) ^ hasItem(?e1,11) ^
hasItem(?e1,17) ^ hasItem(?e1,19) -> hasMinF(?e1, 3) ^ hasMaxF(?e1, 1567841)

Expert rule 4:
ManufacturingProcess(?s) ^ hasEvent(?s, ?e1) ^ hasItem(?e1,2) ^ hasItem(?e1,4) ^ hasItem(?e1,5) ^ hasItem(?e1,7) ^
hasItem(?e1,11) ^ hasItem(?e1,17) ^ hasItem(?e1,19) -> hasMinF(?e1, 1) ^ hasMaxF(?e1, 2656921)

Expert rule 5:
ManufacturingProcess(?s) ^ hasEvent(?s, ?e1) ^ hasItem(?e1,2) ^ hasItem(?e1,4) ^ hasItem(?e1,5) ^ hasItem(?e1,7) ^
hasItem(?e1,11) ^ hasItem(?e1,17) ^ hasItem(?e1,19) -> hasMinF(?e1, 1) ^ hasMaxF(?e1, 2656921)

Expert rule 6:
ManufacturingProcess(?s) ^ hasEvent(?s, ?e1) ^ hasItem(?e1,2) ^ hasItem(?e1,8) ^ hasItem(?e1,9) -> hasMinF(?e1, 115) ^
hasMaxF(?e1, 784613)

Expert rule 7:
ManufacturingProcess(?s) ^ hasEvent(?s, ?e1) ^ hasItem(?e1,2) ^ hasItem(?e1,4) ^ hasItem(?e1,5) ^ hasItem(?e1,7) ^
hasItem(?e1,8) -> hasMinF(?e1, 525) ^ hasMaxF(?e1, 254651)

Expert rule 8:
ManufacturingProcess(?s) ^ hasEvent(?s, ?e1) ^ hasItem(?e1,2) ^ hasItem(?e1,4) ^ hasItem(?e1,6) ^ hasItem(?e1,7) ^
hasItem(?e1,8) -> hasMinF(?e1, 310) ^ hasMaxF(?e1, 421974)

```

Figure 9: An example expert rule base for the UCI SECOM data set.

round, five expert rules are taken as input within the rule integration process. After rule quality issue detection, the updated rule base consists of 6 rules. As a result, 13 rows of prediction results are returned after executing the SQWRL query. They are shown in the table at the bottom part of the failure detection and prediction GUI. In the bottom table, the left two columns are the individuals populated in

Figure 10 shows the 'HALFBACK Prototype: the Failure Prediction Results Interface' window. It displays a table of SQWRL queries and their results. A red arrow points to the SQWRL query for retrieving failure prediction results.

Name	Query	Comment
Q1	hasItem(?t1, 7) ^ hasItem(?t1, 10) ^ hasEvent(?s, ?e1) ^ hasItem(?e1, 4) ^ ManufacturingProcess(?s) ^ hasItem(?e1, 19) ^	
Q2	hasItem(?t1, 7) ^ hasItem(?t1, 10) ^ hasEvent(?s, ?e1) ^ hasItem(?e1, 17) ^ hasEvent(?s, ?e1) ^ hasItem(?e1, 4) ^ hasItem(?e1, 2) ^	
Q3	hasItem(?t1, 7) ^ hasItem(?t1, 10) ^ hasEvent(?s, ?e1) ^ hasItem(?e1, 17) ^ hasEvent(?s, ?e1) ^ hasItem(?e1, 4) ^ ManufacturingProcess(?s) ^	
Q124	hasItem(?t1, 7) ^ hasEvent(?s, ?e1) ^ hasItem(?e1, 17) ^ hasEvent(?s, ?e1) ^ hasItem(?e1, 4) ^ ManufacturingProcess(?s) ^	
Q248	hasItem(?t1, 7) ^ hasEvent(?s, ?e1) ^ hasItem(?e1, 17) ^ hasEvent(?s, ?e1) ^ hasItem(?e1, 4) ^ ManufacturingProcess(?s) ^	
SQWRL query for retrieving failure prediction results	hasEvent(?s, ?e1) ^ hasItem(?e1, 2) ^ hasItem(?e1, 4) ^ hasItem(?e1, 5) ^ hasItem(?e1, 7) ^ hasItem(?e1, 11) ^ hasItem(?e1, 17) ^ hasItem(?e1, 19) -> hasMinF(?e1, 1) ^ hasMaxF(?e1, 2656921)	

SQWRL Queries	COWL 2 RL	SQWRL query for retrieving failure prediction results	Minimum Time to Failure	Maximum Time to Failure
SemiconductorManufacturingProcess_1	Event_15	Event	1	2056921
SemiconductorManufacturingProcess_1	Event_15	Event	1	2056921
SemiconductorManufacturingProcess_1	Event_15	Event	1	1567841
SemiconductorManufacturingProcess_2	Event_16	Event	3	1567841
SemiconductorManufacturingProcess_1	Event_15	Event	3	1567841
SemiconductorManufacturingProcess_1	Event_15	Event	3	1567841
SemiconductorManufacturingProcess_1	Event_15	Event	3	1567841
SemiconductorManufacturingProcess_1	Event_15	Event	3	1567841
SemiconductorManufacturingProcess_1	Event_15	Event	3	1567841
SemiconductorManufacturingProcess_1	Event_15	Event	3	1567841
SemiconductorManufacturingProcess_1	Event_15	Event	3	1567841
SemiconductorManufacturingProcess_2	Event_16	Event	3	1567841
SemiconductorManufacturingProcess_2	Event_16	Event	3	1567841
SemiconductorManufacturingProcess_2	Event_16	Event	3	1567841

Figure 10: The first round of failure detection and prediction. 6 rules are used for failure prediction, SQWRL query returns 13 rows of prediction results.

the MPMO ontology. The right two columns gives the minimum and maximum numeric time values from an event to a future failure. The computation of these two numeric time values are performed by the Clasp-CPM algorithm. These two columns together with the event pair gives the temporal constraints of failures.

After the first round of prediction, we continue integrating more expert rules into the rule base. In the second round, the updated rule base consists of 9 rules, as shown in Fig. 11. Then we use this new rule base to perform the second round of prediction. As a result, we obtain 39 rows of results indicating 39 failures from the SQWRL query. This proves that as more expert rules are integrated into the rule base, the coverage of the whole rule base improves. In this way, the rule base is progressively updated with more expert rules integrated. This enables KSPMI to detect and predict more potential failures and their time of occurrence.

Figure 11: The second round of failure detection and prediction. 9 rules are used for failure prediction, SQWRL query returns 39 rows of prediction results.

5.2.6. Evaluation of rule quality

We then conduct another experimentation to simulate the expert rule integration process and to evaluate the overall quality of the rule base. To do this, the UCI SECOM data set is separated into a training set (80%) and test set (20%). The training set is used to extract chronicle rules and to construct the chronicle rule base and expert rule base. After that, we select a set of chronicle rules that have the highest fitness values. Since these high-fitness chronicle rules are likely to have a higher chance to validate the data sequences in the test set, they are considered as expert rules.

As a start, the frequent chronicle mining approach is applied to the training set. Then the fast non-dominated sorting algorithm [46] is used to obtain a set of best-quality chronicle rules. We then select the chronicle rules within the first Pareto Front. Since the selected rules Pareto-dominate all the rest, we treat them as expert rules. All the rules in the other Pareto Fronts construct a chronicle rule base.

During the expert rule integration process, Algorithm 2 is used to detect redundancy, conflict, and subsumption issues. If no aforementioned issues are detected, the expert rule is integrated into the chronicle rule base. After that, the average fitness of the updated rule base is computed. With each integration, the updated rule base is launched for failure prediction against the test set. Fig. 12 shows the obtained average fitness values with different number of expert rules as input. For the ease of visualization, we choose chronicle support=0.6 and show the change of fitness values. It can be observed that as more expert rules are integrated into the chronicle rule base, the average fitness increases. This is because as more expert rules are integrated into the rule base, there are more chances for the updated rule base to validate the data sequences in the test set. As a result, as more expert rules are launched for reasoning, the integrated rule base has more chances to predict the failures correctly. Thus the overall fitness of the integrated rule base increases.

However, there is one exception where the average fitness of the updated rule base does not increase (number of expert rules = 4). It is because the fitness value of the new

expert rule for the test set is lower than the average value of the whole rule base. Thus the fitness value of the updated rule base is decreased. When the antecedent of the input expert rule only covers a few data sequence in the test set, or most of the temporal constraints of the failures are out of the range that is predicted in the expert rule, the input expert rule shows a low fitness value.

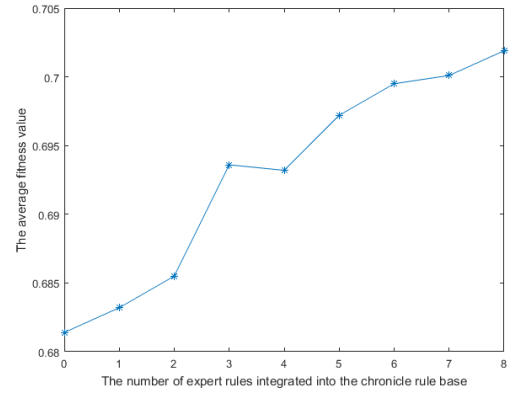


Figure 12: The average fitness values of the rules in the integrated rule base with different number of expert rules.

5.3. Experimentation on synthetic data sets

The performance of KSPMI is also evaluated on several synthetic data sets¹¹. These synthetic data sets are generated following the same format as the one shown in Fig. 4. To apply the Clasp-CPM algorithm for chronicle mining, we tune several parameters to generate different size of synthetic data. These parameters include the number of data sequences in the data set, the size of a single data sequence, the maximum number of items within each event, and the maximum number of items within a single data sequence. Each data sequence in the synthetic data sets is also associated with a generated timestamp.

Same as the experimentation performed on the UCI SECOM data set, the first step in this experimentation is to apply frequent chronicle mining on synthetic data sets. Fig. 13 shows a screenshot of the frequent chronicle mining GUI we obtained from on synthetic data set. Within this data set, the number of data sequences is 1000, and the chronicle support is set to a random number as 0.73 for the Clasp-CPM algorithm. As a result, the maximum frequency we obtained from chronicles is 893, which indicates that this chronicle appeared in 893 data sequences within the whole data set. There are in total 20 chronicles extracted from this data set. For this case study, the integer 0 in chronicles also stands for a failure event.

After frequent chronicle mining, we proceed to SWRL rule generation and pruning. After clicking the “Transform the extracted chronicles into SWRL rules” button at the bottom, the failure chronicles are transformed into predictive

¹¹All used data sets can be found at: <https://sites.google.com/view/kspmi/knowledge-basedsystem/home>



Figure 13: Frequent chronicle mining on one synthetic data set. Chronicle support = 0.73.

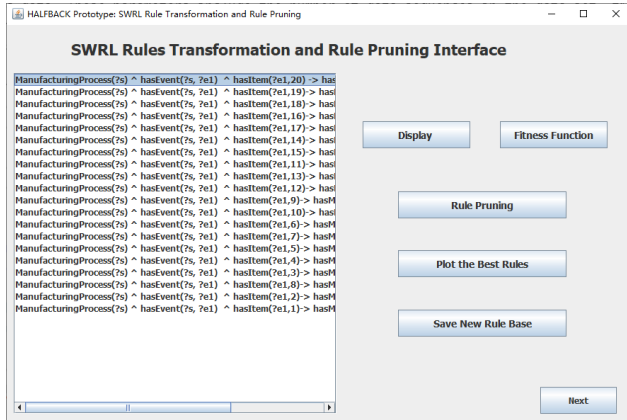


Figure 14: 20 SWRL rules are generated from the chronicles shown in Fig. 13.

SWRL rules. 20 SWRL rules are generated after this step. Fig. 14 shows a screenshot of the generated rules.

We then activate the SWRL rule pruning process. The fast non-dominated sorting algorithm [46] is triggered to remove low-quality rules and select best-quality ones for ontology reasoning. Fig. 17 shows the rule pruning result. For the tested synthetic data set, there is only one SWRL rule that dominates the rest. Thus only this non-dominated rule is plotted on the right side of Fig. 17.

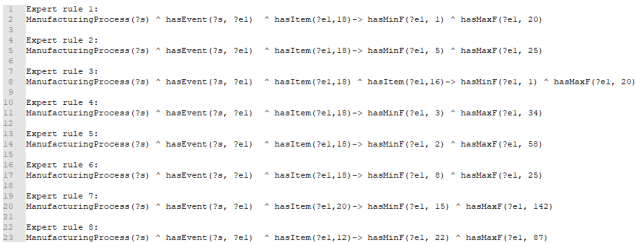


Figure 15: An example expert rule base for the synthetic data set.

After that, the simulated expert rules are integrated into the chronicle rule base. Based on the rule quality definitions given in Section 3.3 and the rule quality issue detection Algorithm 2, KSPMI progressively improve the overall quality

of the rule base. Fig. 15 presents the expert rule base for this synthetic data set.

In the end, after pushing all 8 expert rules into KSPMI, the integrated rule base is launched for failure prediction. For this experimentation, the integrated rule base consists of 5 SWRL rules. After launching the SQWRL query, we obtain 28 rows of results. Fig. 16 shows the prediction results for this synthetic data set. Similar to the experimentation conducted on the UCI SECOM data set, the two table columns on the right side together with the events pairs (events-failures) give the temporal constraints for failure occurrence. Up to this step, we have accomplished the failure prediction task for this synthetic data set.

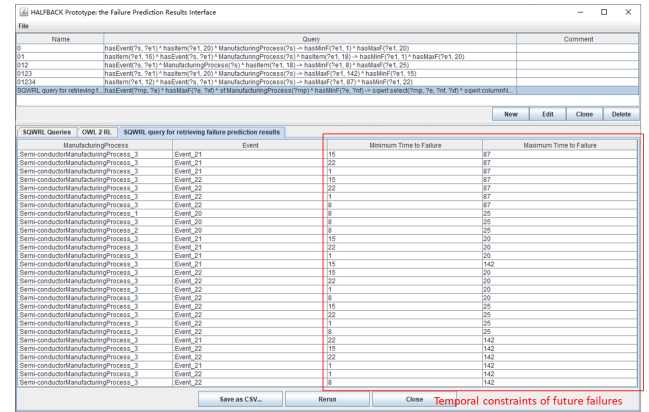


Figure 16: Failure prediction GUI for the synthetic data set. 5 rules are launched for failure prediction, SQWRL query returns 28 rows of prediction results.

6. Conclusions

This paper introduces a knowledge-based predictive maintenance system for Industry 4.0, named KSPMI. KSPMI enables an automatic predictive maintenance process by leveraging both statistical AI and symbolic AI methods. The execution of KSPMI starts with the chronicle mining process on industrial data sets, after which a set of failure chronicles are extracted. Then, a novel hybrid approach is proposed to transform the extracted chronicles into predictive SWRL rules. After that, rule base pruning and integration approaches are proposed to update the rule base automatically and progressively. Simulated expert rules are taken as input to the rule base integration process. At last, the integrated rule base is used together with the MPMO ontology to perform ontology reasoning for the goal of failure prediction. KSPMI has been tested and validated on both real-world and synthetic data sets.

The contributions proposed in this paper may induce potential future research. We describe them as follows:

- The first future work is the consideration of more rule quality measures. We aim to involve more rule quality measures for rule pruning, such as *Association*, *Information*, and *Logical Sufficiency* [3]. Among them, *Association* is a measure for indicating a relationship be-

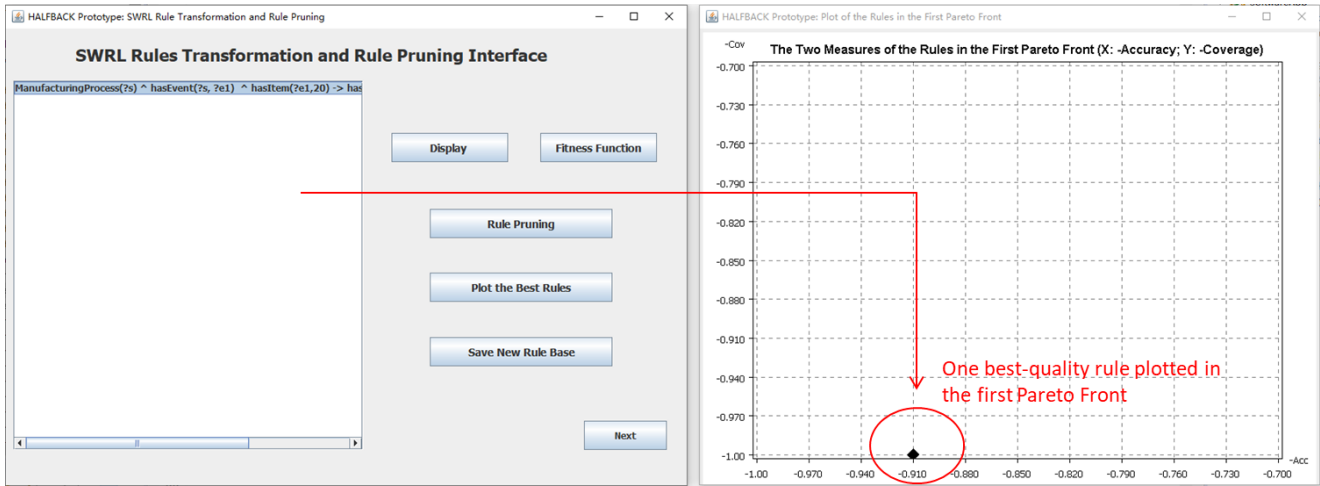


Figure 17: Rule pruning result for one synthetic data set.

tween the classification for the columns and the rows in the contingency table. *Information* measures the amount of entities a R needs to encode for correctly classifying an instance into a certain class. Statistical values such as *Information Score* can be computed for a rule R , to evaluate how much information it can contribute for a classification problem [39]. *Logical Sufficiency* is a standard likelihood ratio statistics that aims to measure whether a rule R is suitable to be used for classifying a certain class.

- The second future work is to enable the evaluation of rule quality for expert rules. For KSPMI, a rule base integration approach is proposed for integrating rules that are obtained from experience capitalisation. However, the proposed approach is based on the hypothesis that the expert rules have a fitness value of 1. Because of this, expert rules are considered more reliable than chronicle rules. Indeed, due to the complex and uncertain characteristics of the manufacturing domain, experts sometimes may provide erroneous expert rules for failure prediction. The erroneous expert rules may lead to incorrect failure prediction. To address this issue, an additional evaluation step is required to examine the performance and quality of expert rules. After examining the quality of expert rules, instead of pruning a subset of low-quality rules merely from the chronicle rule base, KSPMI will perform rule pruning and select a set of best-quality rules from the integrated rule base. This will achieve higher performance for failure prediction.
- The third future work is the capability of the system to handle real-time data. Since the manufacturing domain is highly-dynamic, how to process real-time and heterogeneous data streams on the fly is a crucial concern to manufactures. However, KSPMI uses classical ontology reasoning for failure prediction, within which the generated SWRL rules can deal with highly

dynamic data in a timely fashion. To cope with this challenge, stream reasoning techniques will be considered to reason upon a variety of highly dynamic data [17]. We aim to use continuous queries and rules together with powerful stream reasoners to continuously reason on data streams. In this way, KSPMI will be able to detect and predict machinery failures in real-time.

Acknowledgements

This work has received funding from INTERREG Upper Rhine (European Regional Development Fund) and the Ministries for Research of Baden-Württemberg, Rheinland-Pfalz (Germany) and from the Grand Est French Region in the framework of the Science Offensive Upper Rhine HALFBACK project.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Amruthnath, N., Gupta, T., 2018. A research study on unsupervised machine learning algorithms for early fault detection in predictive maintenance, in: 2018 5th International Conference on Industrial Engineering and Applications (ICIEA), IEEE. pp. 355–361.
- [2] An, A., Cercone, N., 1999. An empirical study on rule quality measures, in: International Workshop on Rough Sets, Fuzzy Sets, Data Mining, and Granular-Soft Computing, Springer. pp. 482–491.
- [3] An, A., Cercone, N., 2001. Rule quality measures for rule induction systems: Description and evaluation. *Computational Intelligence* 17, 409–424.
- [4] Baban, C.F., Baban, M., Suteu, M.D., 2016. Using a fuzzy logic approach for the predictive maintenance of textile machines. *Journal of Intelligent & Fuzzy Systems* 30, 999–1006.
- [5] Bellatreche, L., Dung, N.X., Pierra, G., Hondjack, D., 2006. Contribution of ontology-based data modeling to automatic integration

- of electronic catalogues within engineering databases. *Computers in Industry* 57, 711–724.
- [6] Borissova, D., Mustakarov, I., 2012. An integrated framework of designing a decision support system for engineering predictive maintenance. *International Journal “Information Technologies and Knowledge* 6, 366–376.
 - [7] Butler, K.L., 1996. An expert system based framework for an incipient failure detection and predictive maintenance system, in: *Proceedings of International Conference on Intelligent System Application to Power Systems*, IEEE. pp. 321–326.
 - [8] Cao, Q., Giustozzi, F., Zanni-Merk, C., de Bertrand de Beuvron, F., Reich, C., 2019a. Smart condition monitoring for industry 4.0 manufacturing processes: An ontology-based approach. *Cybernetics and Systems* , 1–15.
 - [9] Cao, Q., Samet, A., Zanni-Merk, C., de Bertrand de Beuvron, F., Reich, C., 2020a. Combining chronicle mining and semantics for predictive maintenance in manufacturing processes. *Semantic Web* , 1–22.
 - [10] Cao, Q., Samet, A., Zanni-Merk, C., de Beuvron, F.d.B., Reich, C., 2019b. An ontology-based approach for failure classification in predictive maintenance using fuzzy c-means and swrl rules. *Procedia Computer Science* 159, 630–639.
 - [11] Cao, Q., Zanni-Merk, C., Reich, C., 2018. Ontologies for manufacturing process modeling: A survey, in: *International Conference on Sustainable Design and Manufacturing*, Springer. pp. 61–70.
 - [12] Cao, Q., Zanni-Merk, C., Samet, A., de Beuvron, F.d.B., Reich, C., 2020b. Using rule quality measures for rule base refinement in knowledge-based predictive maintenance systems. *Cybernetics and Systems* 51, 161–176.
 - [13] Carvalho, T.P., Soares, F.A., Vita, R., Francisco, R.d.P., Basto, J.P., Alcalá, S.G., 2019. A systematic literature review of machine learning methods applied to predictive maintenance. *Computers & Industrial Engineering* 137, 106024.
 - [14] Celaya, J., Saxena, A., Saha, S., Goebel, K.F., 2011. Prognostics of power mosfets under thermal stress accelerated aging using data-driven and model-based methodologies .
 - [15] Cram, D., Mathern, B., Mille, A., 2012. A complete chronicle discovery approach: application to activity analysis. *Expert Systems* 29, 321–346.
 - [16] Dalenogare, L.S., Benitez, G.B., Ayala, N.F., Frank, A.G., 2018. The expected contribution of industry 4.0 technologies for industrial performance. *International Journal of Production Economics* 204, 383–394.
 - [17] Dell’Aglia, D., Della Valle, E., van Harmelen, F., Bernstein, A., 2017. Stream reasoning: A survey and outlook. *Data Science* 1, 59–83.
 - [18] Domingue, J., Fensel, D., Hendler, J.A., 2011. *Handbook of semantic web technologies*. Springer Science & Business Media.
 - [19] Dragomir, O.E., Gouriveau, R., Dragomir, F., Minca, E., Zerhouni, N., 2009. Review of prognostic problem in condition-based maintenance, in: *2009 European Control Conference (ECC)*, IEEE. pp. 1587–1592.
 - [20] Efthymiou, K., Papakostas, N., Mourtzis, D., Chrysosouris, G., 2012. On a predictive maintenance platform for production systems. *Procedia CIRP* 3, 221–226.
 - [21] Faiz, R., Edirisinghe, E.A., 2009. Decision making for predictive maintenance in asset information management. *Interdisciplinary Journal of Information, Knowledge, and Management* 4, 23–36.
 - [22] Gangemi, A., 2005. Ontology design patterns for semantic web content, in: *International semantic web conference*, Springer. pp. 262–276.
 - [23] Gohel, H.A., Upadhyay, H., Lagos, L., Cooper, K., Sanzetenea, A., 2020. Predictive maintenance architecture development for nuclear infrastructure using machine learning. *Nuclear Engineering and Technology* 52, 1436–1442.
 - [24] Gómez-Pérez, A., Corcho, O., 2002. Ontology languages for the semantic web. *IEEE Intelligent systems* 17, 54–60.
 - [25] Grall, A., Dieulle, L., Béranger, C., Roussignol, M., 2002. Continuous-time predictive-maintenance scheduling for a deteriorating system. *IEEE transactions on reliability* 51, 141–150.
 - [26] Grimm, S., 2009. *Semantic matchmaking with nonmonotonic description logics*. volume 1. IOS Press.
 - [27] Gruber, T., 2009. *Ontology*. Springer.
 - [28] Gruber, T.R., 1993. A translation approach to portable ontology specifications. *Knowledge acquisition* 5, 199–220.
 - [29] Gustafsson, F., Gunnarsson, F., Bergman, N., Forssell, U., Jansson, J., Karlsson, R., Nordlund, P.J., 2002. Particle filters for positioning, navigation, and tracking. *IEEE Transactions on signal processing* 50, 425–437.
 - [30] Guyon, I., Elisseeff, A., 2003. An introduction to variable and feature selection. *Journal of machine learning research* 3, 1157–1182.
 - [31] Heng, A., Zhang, S., Tan, A.C., Mathew, J., 2009. Rotating machinery prognostics: State of the art, challenges and opportunities. *Mechanical systems and signal processing* 23, 724–739.
 - [32] Hesser, D.F., Markert, B., 2019. Tool wear monitoring of a retrofitted cnc milling machine using artificial neural networks. *Manufacturing letters* 19, 1–4.
 - [33] Horrocks, I., Patel-Schneider, P.F., Boley, H., Tabet, S., Grosz, B., Dean, M., et al., 2004. Swrl: A semantic web rule language combining owl and ruleml. *W3C Member submission* 21, 1–31.
 - [34] Javed, K., Gouriveau, R., Zemouri, R., Zerhouni, N., 2011. Improving data-driven prognostics by assessing predictability of features.
 - [35] Javed, K., Gouriveau, R., Zerhouni, N., 2017. State of the art and taxonomy of prognostics approaches, trends of prognostics applications and open issues towards maturity at different technology readiness levels. *Mechanical Systems and Signal Processing* 94, 214–236.
 - [36] Kallianpur, G., 2013. *Stochastic filtering theory*. volume 13. Springer Science & Business Media.
 - [37] Kaparthy, S., Bumlauskas, D., 2020. Designing predictive maintenance systems using decision tree-based machine learning techniques. *International Journal of Quality & Reliability Management* .
 - [38] Kingma, D.P., Mohamed, S., Rezende, D.J., Welling, M., 2014. Semi-supervised learning with deep generative models, in: *Advances in neural information processing systems*, pp. 3581–3589.
 - [39] Kononenko, I., Bratko, I., 1991. Information-based evaluation criterion for classifier’s performance. *Machine Learning* 6, 67–80.
 - [40] Langone, R., Alzate, C., De Ketelaere, B., Vlasselaer, J., Meert, W., Suykens, J.A., 2015. Ls-svm based spectral clustering and regression for predicting maintenance of industrial machines. *Engineering Applications of Artificial Intelligence* 37, 268–278.
 - [41] Lee, J., Bagheri, B., Kao, H.A., 2015. A cyber-physical systems architecture for industry 4.0-based manufacturing systems. *Manufacturing letters* 3, 18–23.
 - [42] Liu, C., Gryllias, K., 2020. A semi-supervised support vector data description-based fault detection method for rolling element bearings based on cyclic spectral analysis. *Mechanical Systems and Signal Processing* 140, 106682.
 - [43] Macek, J., Zak, Z., Vitek, O., 2015. Physical model of a twin-scroll turbine with unsteady flow. *Technical Report*. SAE Technical Paper.
 - [44] Maleki, E., Belkadi, F., Boli, N., Van Der Zwaag, B.J., Alexopoulos, K., Koukas, S., Marin-Perianu, M., Bernard, A., Mourtzis, D., 2018. Ontology-based framework enabling smart product-service systems: Application of sensing systems for machine health monitoring. *IEEE Internet of Things Journal* 5, 4496–4505.
 - [45] McCann, M., Johnston, A., 2008. Secom dataset, uci machine learning repository.
 - [46] Ortega, G., Filatovas, E., Garzón, E., Casado, L.G., 2017. Non-dominated sorting procedure for pareto dominance ranking on multicore cpu and/or gpu. *Journal of Global Optimization* 69, 607–627.
 - [47] Ouyang, Z., Sun, X., Chen, J., Yue, D., Zhang, T., 2018. Multi-view stacking ensemble for power consumption anomaly detection in the context of industrial internet of things. *IEEE Access* 6, 9623–9631.
 - [48] Panetto, H., Dassisti, M., Tursi, A., 2012. Onto-pdm: Product-driven ontology for product data management interoperability within manufacturing process environment. *Advanced Engineering Informatics* 26, 334–348.
 - [49] Paolanti, M., Romeo, L., Felicetti, A., Mancini, A., Frontoni, E., Loncarski, J., 2018. Machine learning approach for predictive maintenance

- nance in industry 4.0, in: 2018 14th IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA), IEEE. pp. 1–6.
- [50] Pecht, M., 2009. Prognostics and health management of electronics. Encyclopedia of Structural Health Monitoring .
- [51] Pecht, M.G., 2010. A prognostics and health management roadmap for information and electronics-rich systems. IEICE ESS Fundamentals Review 3, 4_25–4_32.
- [52] Peng, Y., Dong, M., Zuo, M.J., 2010. Current status of machine prognostics in condition-based maintenance: a review. The International Journal of Advanced Manufacturing Technology 50, 297–313.
- [53] Ramírez-Gallego, S., García, S., Mourño-Talín, H., Martínez-Rego, D., Bolón-Canedo, V., Alonso-Betanzos, A., Benítez, J.M., Herrera, F., 2016. Data discretization: taxonomy and big data challenge. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery 6, 5–21.
- [54] Reynolds, D., 2015. Gaussian mixture models. Encyclopedia of biometrics , 827–832.
- [55] Roemer, M.J., Nwadiogbu, E., Bloor, G., 2001. Development of diagnostic and prognostic technologies for aerospace health management applications, in: 2001 IEEE Aerospace Conference Proceedings (Cat. No. 01TH8542), IEEE. pp. 3139–3147.
- [56] Saxena, A., Celaya, J.R., Roychoudhury, I., Saha, S., Saha, B., Goebel, K., 2012. Designing data-driven battery prognostic approaches for variable loading profiles: Some lessons learned, in: European conference of prognostics and health management society, pp. 72–732.
- [57] Sellami, C., Miranda, C., Samet, A., Tobji, M.A.B., de Beuvron, F., 2019. On mining frequent chronicles for machine failure prediction. Journal of Intelligent Manufacturing , 1–17.
- [58] Sellami, C., Miranda, C., Samet, A., Tobji, M.A.B., de Beuvron, F., 2020. On mining frequent chronicles for machine failure prediction. Journal of Intelligent Manufacturing 31, 1019–1035.
- [59] Sellami, C., Samet, A., Tobji, M.A.B., 2018. Frequent chronicle mining: Application on predictive maintenance, in: 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), IEEE. pp. 1388–1393.
- [60] Sikorska, J., Hodkiewicz, M., Ma, L., 2011. Prognostic modelling options for remaining useful life estimation by industry. Mechanical systems and signal processing 25, 1803–1836.
- [61] Simões, A., Viegas, J.M., Farinha, J.T., Fonseca, I., 2017. The state of the art of hidden markov models for predictive maintenance of diesel engines. Quality and Reliability Engineering International 33, 2765–2779.
- [62] Susto, G.A., Schirru, A., Pampuri, S., McLoone, S., Beghi, A., 2014. Machine learning for predictive maintenance: A multiple classifier approach. IEEE Transactions on Industrial Informatics 11, 812–820.
- [63] Uckun, S., Goebel, K., Lucas, P.J., 2008. Standardizing research methods for prognostics, in: 2008 International Conference on Prognostics and Health Management, IEEE. pp. 1–10.
- [64] Von Birgelen, A., Buratti, D., Mager, J., Niggemann, O., 2018. Self-organizing maps for anomaly localization and predictive maintenance in cyber-physical production systems. Procedia CIRP 72, 480–485.
- [65] Wang, K., 2016. Intelligent predictive maintenance (ipdm) system–industry 4.0 scenario. WIT Transactions on Engineering Sciences 113, 259–268.
- [66] Wang, Z., Chen, C.H., Zheng, P., Li, X., Khoo, L.P., 2019. A graph-based context-aware requirement elicitation approach in smart product-service systems. International Journal of Production Research , 1–17.
- [67] Yan, X., Han, J., Afshar, R., 2003. Closan: Mining: Closed sequential patterns in large datasets, in: Proceedings of the 2003 SIAM international conference on data mining, SIAM. pp. 166–177.
- [68] Yoon, A.S., Lee, T., Lim, Y., Jung, D., Kang, P., Kim, D., Park, K., Choi, Y., 2017. Semi-supervised learning with deep generative models for asset failure prediction. arXiv preprint arXiv:1709.00845 .
- [69] Zhou, K., Liu, T., Zhou, L., 2015. Industry 4.0: Towards future industrial opportunities and challenges, in: 2015 12th International conference on fuzzy systems and knowledge discovery (FSKD), IEEE. pp. 2147–2152.
- [70] Zhu, M., Zhu, H., Guo, F., Chen, X., Ju, J.W., 2021. Tunnel condition assessment via cloud model-based random forests and self-training approach. Computer-Aided Civil and Infrastructure Engineering 36, 164–179.